

搭建这个智能体，请严格按照以下 5 个阶段推进。**现在立刻从第一步开始**，不要往后看。

第一阶段：跑通单次闭环（现在开始，只做这 4 件事）

1. **写死 System Prompt**：给大模型一个明确身份，规定它只能通过工具与外界交互，禁止瞎编代码结果。
2. **实现 ReAct 核心循环**：写一个 while 循环，处理 用户输入 -> LLM 思考 -> 调用工具 -> 返回结果 -> LLM 继续思考 的流转。
3. **硬编码 3 个工具**：
 - read_file (读文件)
 - write_file (写文件)
 - run_command (执行终端命令)
4. **终端打印日志**：把 LLM 的思考过程和工具执行结果直接 print 出来，方便你看到它在干嘛。

▮ **阶段验收标准**：在终端输入“帮我新建一个 hello.py 并运行它”，智能体能自动调用 write_file 写入代码，调用 run_command 执行，并返回执行结果。

第二阶段：增强上下文与规划能力（闭环跑通后加上）

1. **引入 TODOlist**：让 LLM 在动手改代码前，先输出一个 TODO 列表（存在上下文里），每完成一项自己打个勾，防止改着改着忘事。
2. **短期记忆管理**：实现基本的对话历史截断机制，防止 Token 爆框。
3. **上下文管理（AST 级别）**：当 read_file 读取大文件时，实现按函数/类级别的片段读取，而不是整个文件塞进去。

▮ **阶段验收标准**：给智能体一个跨越 3 个文件的重构任务，它能列出 1-2-3 步骤，并按步骤依次修改，不会因为上下文太长而崩溃。

第三阶段：安全隔离与协议标准化（具备基本可用性后加上）

1. **沙箱机制**：将 run_command 放到 Docker 容器或安全沙箱中执行，限制网络访问和文件系统写权限，防止 LLM 误删系统文件。
2. **工具调用升级为 MCP**：把硬编码的 3 个工具剥离出来，接入 MCP (Model Context Protocol) 协议，让智能体可以动态发现和使用工具。
3. **Hook 机制**：在工具执行前后加入钩子。例如：write_file 执行前自动触发 Git 备份，run_command 执行后自动触发格式检查。

▮ **阶段验收标准**：智能体可以在隔离环境中执行不可信代码，并且你能通过配置文件动态给它添加新工具（如浏览器工具），而不需要改核心代码。

第四阶段：拆分与协作（单 Agent 遇到瓶颈后加上）

1. **SubAgent（子智能体）**：将主循环拆分为 Router 和 Worker。主 Agent 负责规划，遇到具体编码任务，派生一个 SubAgent 去执行，执行完把结果汇总回来。
2. **Agent Team（多角色团队）**：引入不同角色的 Agent（如：架构师、程序员、测试员），他们共享一个 TODOlist，按流程流转任务。

▮ **阶段验收标准**：输入“开发一个贪吃蛇游戏”，架构师输出设计，程序员写代码，测试员自动写测试用例并运行，最终主 Agent 返回成功结果。

第五阶段：工程化与高级特性（准备发布/开源时加上）

1. **Worktree（并行开发）**：结合 Git Worktree，当主分支有紧急 Bug 需要修，同时又有新特性在开发时，智能体能在两个独立目录并行工作，互不干扰。
2. **可观测性与评测（Eval）**：接入 LangSmith/Phoenix 等专业追踪平台，记录每一步的耗时和 Token 消耗；构建代码编辑的基准测试集，自动化评测智能体的编码成功率。

▮ **阶段验收标准**：你能看着 Dashboard 准确分析出智能体在哪里卡住了，且每一次代码迭代都有量化的质量评估。

现在的行动指令：打开你的 IDE，新建一个文件，开始写第一阶段的 ReAct While 循环和那 3 个工具函数。

注:本文部分内容由AI生成,无法确保真实准确,仅供参考